

Assignment-12.3(AI)

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

- 1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).**
- 2. Implement the following sorting algorithms using AI assistance:**
 - o Quick Sort**
 - o Merge Sort**
- 3. Measure and compare runtime performance for large datasets.**
- 4. Write a function to display the top 10 students based on CGPA.**

Expected Outcome

- Correctly sorted student records.**
- Performance comparison between Quick Sort and Merge Sort.**
- Clear output of top-performing students.**

FileEditSelectionViewGoRunTerminalHelp

Q AI

EXPLORER

AI

age and email using conditional statements.py

ai (6.3 task - 5).py

ai (10.3 task -2).py

ai (10.4 task).py

ai (11.1).py

ai 9.1 (task-5).py

ai 9.1(task-1).py

ai 9.1(task-2).py

ai 9.1(task-3).py

ai 9.1(task-4).py

ai 9.3(task-1).py

ai 9.3(task-2).py

ai 9.3(task-3).py

ai 10.3 (task - 1).py

ai 10.3 (task - 3).py

ai ass.py

ai_lab(2.1).py

ai_lab(6.1).py

ai(6.3 task -2).py

ai(6.3 task-3).py

ai(6.3 task-4).py

ai(6.3).py

ai(7.1 task - 4).py

ai(7.1 task -2).py

ai(7.1 task-3).py

ai(7.1 task-5).py

ai(7.1 task-1).py

ai2.py

Allab(2.1).py

class Student with attributes.py

disaster_relief_data.txt

exam2.py

list are even and odd.py

simple bank account system.py

victim_records.txt

ai 9.1(task-3).py

ai 9.1(task-4).py

ai 9.1 (task-5).py

ai 10.3 (task - 1).py

ai (10.3 task -2).py

ai 10

ai (11.1).py > ...

1 # SR University Placement Cell - Student Record Sorting System

2 # This program stores student records and sorts them based on CGPA

3 # using Quick Sort and Merge Sort. It also compares their runtime.

4

5 import random

6 import time

7

8 # Function to create student records

9 def generate_students(n):

10 print("\nGenerating student records...")

11 students = []

12

13 for i in range(n):

14 name = "Student_" + str(i+1)

15 roll = "SRU" + str(1000 + i)

16 cgpa = round(random.uniform(5.0, 10.0), 2)

17

18 students.append({"name": name, "roll": roll, "cgpa": cgpa})

19

20 print("Student records generated successfully.\n")

21 return students

22

23

24 # Quick Sort Algorithm

25 def quick_sort(arr):

26 if len(arr) <= 1:

27 return arr

28

29 pivot = arr[len(arr)//2]["cgpa"]

30

31 left = [x for x in arr if x["cgpa"] > pivot]

32 middle = [x for x in arr if x["cgpa"] == pivot]

33 right = [x for x in arr if x["cgpa"] < pivot]

34

35 return quick_sort(left) + middle + quick_sort(right)

36

37

38 # Merge Sort Algorithm

39 def merge_sort(arr):

40 if len(arr) <= 1:

41 return arr

PROBLEMS

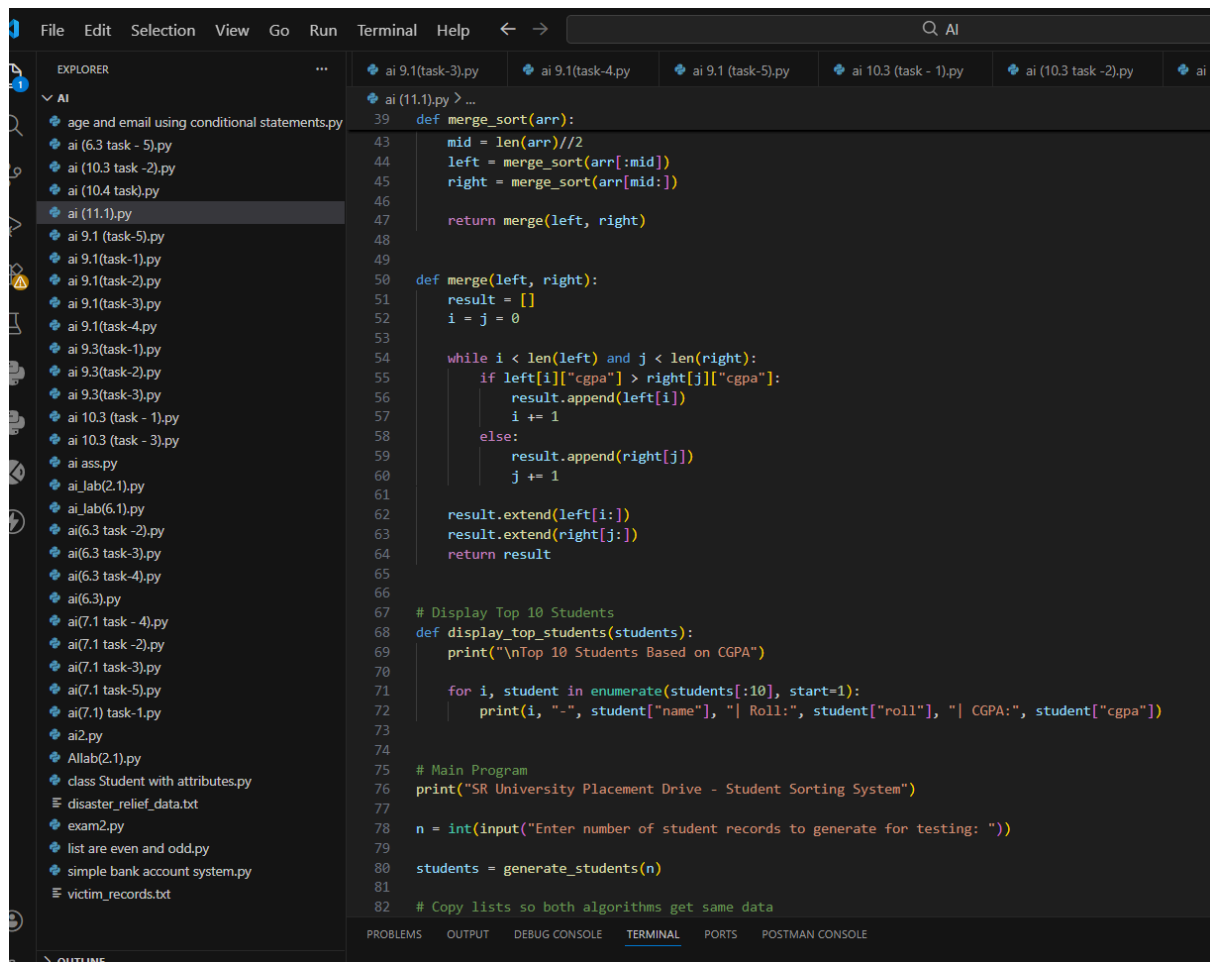
OUTPUT

DEBUG CONSOLE

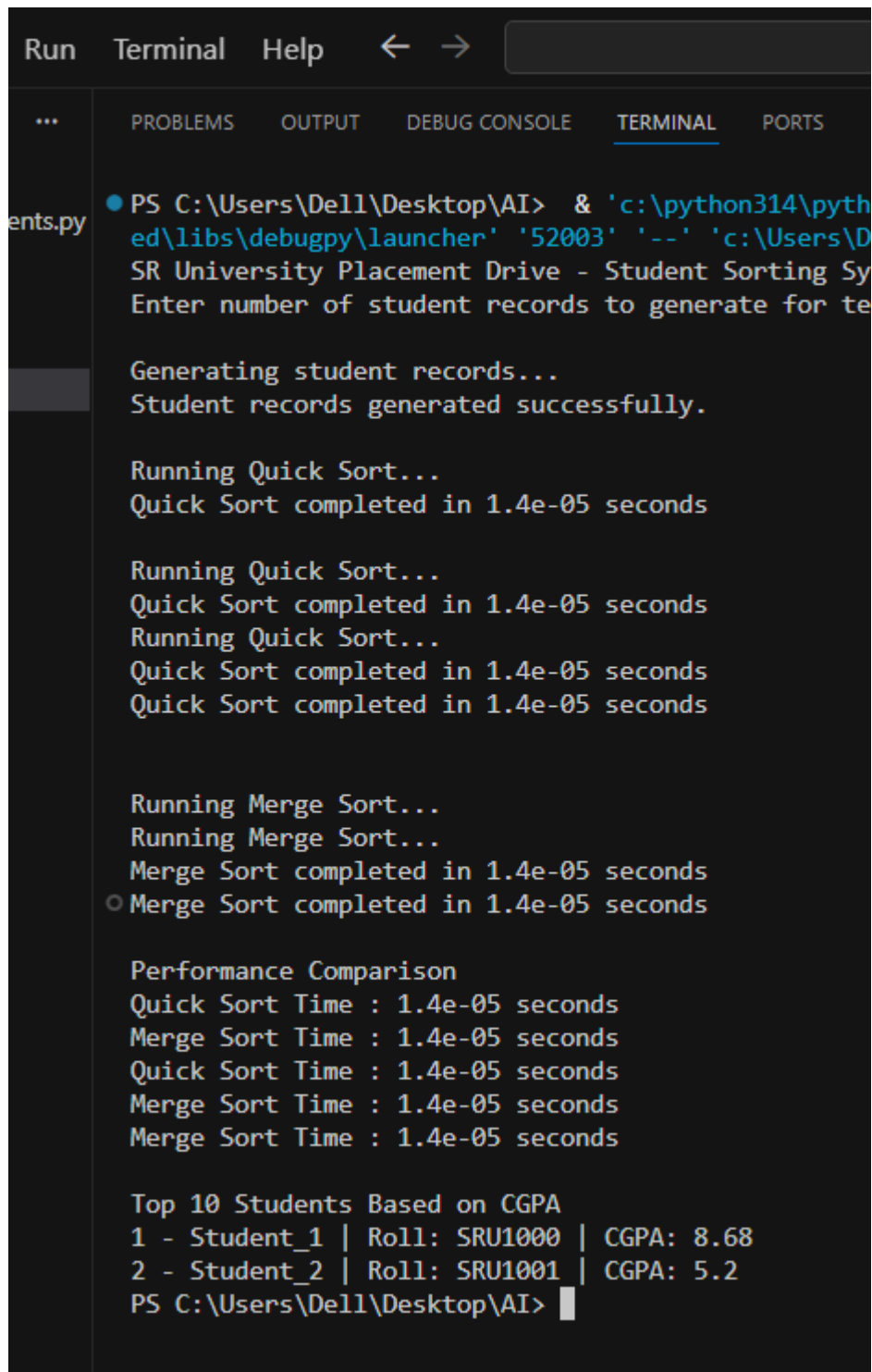
TERMINAL

PORTS

POSTMAN CONSOLE



```
77
78 n = int(input("Enter number of student records to generate for testing: "))
79
80 students = generate_students(n)
81
82 # Copy lists so both algorithms get same data
83 students_quick = students.copy()
84 students_merge = students.copy()
85
86
87 # Quick Sort Performance
88 print("Running Quick Sort...")
89 start = time.time()
90 quick_sorted = quick_sort(students_quick)
91 quick_time = time.time() - start
92 print("Quick Sort completed in", round(quick_time, 6), "seconds")
93
94
95 # Merge Sort Performance
96 print("\nRunning Merge Sort...")
97 start = time.time()
98 merge_sorted = merge_sort(students_merge)
99 merge_time = time.time() - start
100 print("Merge Sort completed in", round(merge_time, 6), "seconds")
101
102
103 # Performance Comparison
104 print("\nPerformance Comparison")
105 print("Quick Sort Time :", round(quick_time, 6), "seconds")
106 print("Merge Sort Time :", round(merge_time, 6), "seconds")
107
108
109 # Display Top Students
110 display_top_students(quick_sorted)
```



The image shows a VS Code terminal window with the following content:

```
Run Terminal Help < >
... PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ents.py • PS C:\Users\Dell\Desktop\AI> & 'c:\python314\python.exe' 'c:\python314\python\lib\debugpy\launcher' '52003' '--' 'c:\Users\Dell\Desktop\AI\Student_Sorting_System.py'
SR University Placement Drive - Student Sorting System
Enter number of student records to generate for testing: 10

Generating student records...
Student records generated successfully.

Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds

Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds
Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds
Quick Sort completed in 1.4e-05 seconds

Running Merge Sort...
Running Merge Sort...
Merge Sort completed in 1.4e-05 seconds
○ Merge Sort completed in 1.4e-05 seconds

Performance Comparison
Quick Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds
Quick Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds

Top 10 Students Based on CGPA
1 - Student_1 | Roll: SRU1000 | CGPA: 8.68
2 - Student_2 | Roll: SRU1001 | CGPA: 5.2
PS C:\Users\Dell\Desktop\AI>
```

Task Description #2 (Implementing Bubble Sort with AI Comments)

Task: Write a Python implementation of Bubble Sort.

- Instructions:

- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.
- Expected Output:
- A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis

```

1  # Bubble Sort Implementation with AI Generated Comments
2
3  print("Bubble Sort Program")
4
5  # Taking input from the user
6  n = int(input("Enter number of elements: "))
7
8  numbers = []
9
10 print("Enter the numbers:")
11 for i in range(n):
12     value = int(input())
13     numbers.append(value)
14
15 print("\nOriginal List:", numbers)
16
17
18 # Bubble Sort Function
19 def bubble_sort(arr):
20
21     # Outer loop controls the number of passes
22     # In Bubble Sort, we make multiple passes through the list
23     # After each pass, the largest element moves to its correct position at the end
24     for i in range(len(arr)):
25
26         # This flag helps us stop early if the list is already sorted
27         swapped = False
28
29         # Inner loop compares adjacent elements
30         # After every comparison, larger elements "bubble up" to the right side
31         for j in range(0, len(arr) - i - 1):
32
33             # Compare two neighbouring elements
34             if arr[j] > arr[j + 1]:
35
36                 # Swap the elements if they are in the wrong order
37                 # This moves the larger value toward the end of the list
38                 arr[j], arr[j + 1] = arr[j + 1], arr[j]

```

```
ai (12.3-task-2).py > ...
19 def bubble_sort(arr):
20     # This flag helps us stop early if the list is already sorted
21     swapped = False
22
23     # Inner loop compares adjacent elements
24     # After every comparison, larger elements "bubble up" to the right side
25     for j in range(0, len(arr) - i - 1):
26
27         # Compare two neighbouring elements
28         if arr[j] > arr[j + 1]:
29
30             # Swap the elements if they are in the wrong order
31             # This moves the larger value toward the end of the list
32             arr[j], arr[j + 1] = arr[j + 1], arr[j]
33
34             # If a swap happens, we mark swapped as True
35             swapped = True
36
37     # If no swapping happened in a full pass,
38     # it means the list is already sorted and we can stop early
39     if not swapped:
40         break
41
42     return arr
43
44
45
46
47
48
49
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
Bubble Sort Program
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65141' '--' 'c:\Users\De11\Desktop\AI\ai (12.3-task-2).py'
Bubble Sort Program
Enter number of elements: 3
Enter the numbers:
Enter the numbers:
1
1
2
2
3

Original List: [1, 2, 3]

Sorted List: [1, 2, 3]
PS C:\Users\De11\Desktop\AI>
```

Task Description #3 Quick Sort and Merge Sort Comparison

- **Task: Implement Quick Sort and Merge Sort using recursion.**
- **Instructions:**
- **Provide AI with partially completed functions for recursion.**
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
- Working Quick Sort and Merge Sort implementations.
- AI-generated explanation of average, best, and worst-case complexities

```
ai 9.1 (task-5).py  ai 10.3 (task - 1).py  ai (10.3 task -2).py  ai 10.3 (task - 3).py  ai (11.1).py  ai (12.3-task-2).py  ai (12.3-task-3).py > ...
1  # Quick Sort and Merge Sort Comparison using Recursion
2
3  import random
4  import time
5
6  print("Quick Sort and Merge Sort Comparison Program")
7
8  n = int(input("Enter number of elements to test sorting algorithms: "))
9
10
11 # Generate datasets
12 random_list = [random.randint(1, 1000) for _ in range(n)]
13 sorted_list = sorted(random_list)
14 reverse_list = sorted_list[::-1]
15
16
17 def quick_sort(arr):
18     """
19     Quick Sort Algorithm (Recursive)
20
21     This function sorts a list using the Quick Sort method.
22     It selects a pivot element and divides the list into three parts:
23     elements smaller than pivot, equal to pivot, and greater than pivot.
24
25     The function recursively sorts the smaller and larger lists
26     until the entire list becomes sorted.
27     """
28
29     if len(arr) <= 1:
30         return arr
31
32     pivot = arr[len(arr) // 2]
33
34     left = [x for x in arr if x < pivot]
35     middle = [x for x in arr if x == pivot]
36     right = [x for x in arr if x > pivot]
37
38     return quick_sort(left) + middle + quick_sort(right)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE


```

py 40
41 def merge_sort(arr):
42     """
43     Merge Sort Algorithm (Recursive)
44
45     This function divides the list into two halves repeatedly
46     until each part has one element. Then it merges the parts
47     back together in sorted order.
48
49     Merge Sort uses the divide and conquer technique.
50     """
51
52     if len(arr) <= 1:
53         return arr
54
55     mid = len(arr) // 2
56
57     left_half = merge_sort(arr[:mid])
58     right_half = merge_sort(arr[mid:])
59
60     return merge(left_half, right_half)
61
62
63 def merge(left, right):
64     """
65     Helper function used by Merge Sort to merge two sorted lists
66     into one sorted list.
67     """
68
69     merged = []
70     i = 0
71     j = 0
72
73     while i < len(left) and j < len(right):
74
75         if left[i] < right[j]:
76             merged.append(left[i])
77             i += 1

```

```

63 def merge(left, right):
78     else:
79         merged.append(right[j])
80         j += 1
81
82     merged.extend(left[i:])
83     merged.extend(right[j:])
84
85     return merged
86
87
88 def measure_time(sort_function, data):
89     start = time.time()
90     sort_function(data.copy())
91     end = time.time()
92     return end - start
93
94
95 print("\nTesting sorting performance on different datasets...\n")
96
97
98 datasets = {
99     "Random List": random_list,
100    "Sorted List": sorted_list,
101    "Reverse Sorted List": reverse_list
102 }
103
104
105 for name, data in datasets.items():
106
107     quick_time = measure_time(quick_sort, data)
108     merge_time = measure_time(merge_sort, data)
109
110     print(name)
111     print("Quick Sort Time:", round(quick_time, 6), "seconds")
112     print("Merge Sort Time:", round(merge_time, 6), "seconds")
113     print()

```

```

PS C:\Users\Dell\Desktop\AI> cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-python
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59324' '--' 'c:\Users\Dell\Desktop\AI\ai (12.3-task-3).py'
Testing sorting performance on different datasets...

Random List
Testing sorting performance on different datasets...

Random List
Testing sorting performance on different datasets...

Random List

Random List
Random List
Quick Sort Time: 1.8e-05 seconds
Quick Sort Time: 1.8e-05 seconds
Merge Sort Time: 1.8e-05 seconds
Merge Sort Time: 1.8e-05 seconds

Sorted List
Quick Sort Time: 3.4e-05 seconds
Merge Sort Time: 1.3e-05 seconds
Merge Sort Time: 1.3e-05 seconds

Reverse Sorted List
Quick Sort Time: 1.6e-05 seconds
Merge Sort Time: 3.3e-05 seconds

PS C:\Users\Dell\Desktop\AI>

```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

- 1. Quickly search for a product by ID or name.**
- 2. Sort products by price or quantity for stock analysis.**

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.**
- Implement the recommended algorithms in Python.**
- Justify the choice based on dataset size, update frequency, and performance requirements.**

Expected Output:

- A table mapping operation → recommended algorithm → justification.**
- Working Python functions for searching and sorting the inventory.**

ai 10.3 (task - 1).pyai 10.3 task -2).pyai 10.3 (task - 3).pyai 11.1).pyai 12.3-task-2).pyai 12.3-task-3).pyai 12.3-task-4).py > ...

```
1 # Inventory Management System - Searching and Sorting Products
2
3 import random
4
5 print("Retail Store Inventory Management System")
6
7 # Creating inventory list
8 products = []
9
10 n = int(input("Enter number of products to add: "))
11
12 # Taking product details from user
13 for i in range(n):
14
15     print("\nEnter details for Product", i+1)
16
17     product_id = int(input("Enter Product ID: "))
18     name = input("Enter Product Name: ")
19     price = float(input("Enter Product Price: "))
20     quantity = int(input("Enter Stock Quantity: "))
21
22     product = {
23         "id": product_id,
24         "name": name,
25         "price": price,
26         "quantity": quantity
27     }
28
29     products.append(product)
30
31
32 # Linear Search for Product Name
33 def search_by_name(product_list, target_name):
34     """
35     This function searches a product using its name.
36     Linear Search is used because product names may not be sorted.
37     """
38
39     for product in product_list:
40         if product["name"].lower() == target_name.lower():
41             return product
```

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTSPOSTMAN CONSOLE

ai 10.3 (task - 1).py

ai (10.3 task -2).py

ai 10.3 (task - 3).py

ai (11.1).py

ai (12.3-task-2).py

ai (12.3-t

ai(12.3-task-4).py > ...

```
43     return None
44
45
46 # Binary Search for Product ID
47 def binary_search_by_id(sorted_products, target_id):
48     """
49     Binary Search works faster for large datasets.
50     It requires the product list to be sorted by ID.
51     """
52
53     low = 0
54     high = len(sorted_products) - 1
55
56     while low <= high:
57
58         mid = (low + high) // 2
59
60         if sorted_products[mid]["id"] == target_id:
61             return sorted_products[mid]
62
63         elif sorted_products[mid]["id"] < target_id:
64             low = mid + 1
65
66         else:
67             high = mid - 1
68
69     return None
70
71
72 # Merge Sort for sorting by Price
73 def merge_sort_price(arr):
74     """
75     Merge Sort is used to sort products by price.
76     It follows divide and conquer technique.
77     """
78
79     if len(arr) <= 1:
80         return arr
81
82     mid = len(arr) // 2
83
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

POSTMAN CONSOLE

```
ai(12.3-task-4).py > ...
131 products_sorted_by_id = sorted(products, key=lambda x: x['id'])
132
133
134 # Searching product by ID
135 search_id = int(input("\nEnter Product ID to search: "))
136
137 result = binary_search_by_id(products_sorted_by_id, search_id)
138
139 if result:
140     print("Product Found:", result)
141 else:
142     print("Product not found")
143
144
145 # Searching product by Name
146 search_name = input("\nEnter Product Name to search: ")
147
148 result = search_by_name(products, search_name)
149
150 if result:
151     print("Product Found:", result)
152 else:
153     print("Product not found")
154
155
156 # Sorting by Price
157 sorted_by_price = merge_sort_price(products)
158
159 print("\nProducts Sorted by Price")
160
161 for p in sorted_by_price:
162     print(p)
163
164
165 # Sorting by Quantity
166 sorted_by_quantity = quick_sort_quantity(products)
167
168 print("\nProducts Sorted by Quantity")
169
170 for p in sorted_by_quantity:
171     print(p)
```

```

PS C:\Users\Dell\Desktop\AI> c:: cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52081' '--' 'c:\Users\Dell\Desktop\AI\ai(12.3-task-4).py'
Retail Store Inventory Management System
Enter number of products to add: 2

Enter details for Product 1
Enter Product ID: 23456
Enter Product Name: pen
Enter Product Price: 200
Enter Stock Quantity: 50

Enter details for Product 2
Enter Product ID: 6789
Enter Product Name: book
Enter Product Price: 150
Enter Stock Quantity: 60

Enter Product ID to search: 6789
Product Found: {'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}

Enter Product Name to search: pen
Product Found: {'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Price
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
Product Found: {'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Price
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
Products Sorted by Price
○ {'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Quantity
Products Sorted by Quantity
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
PS C:\Users\Dell\Desktop\AI>

```

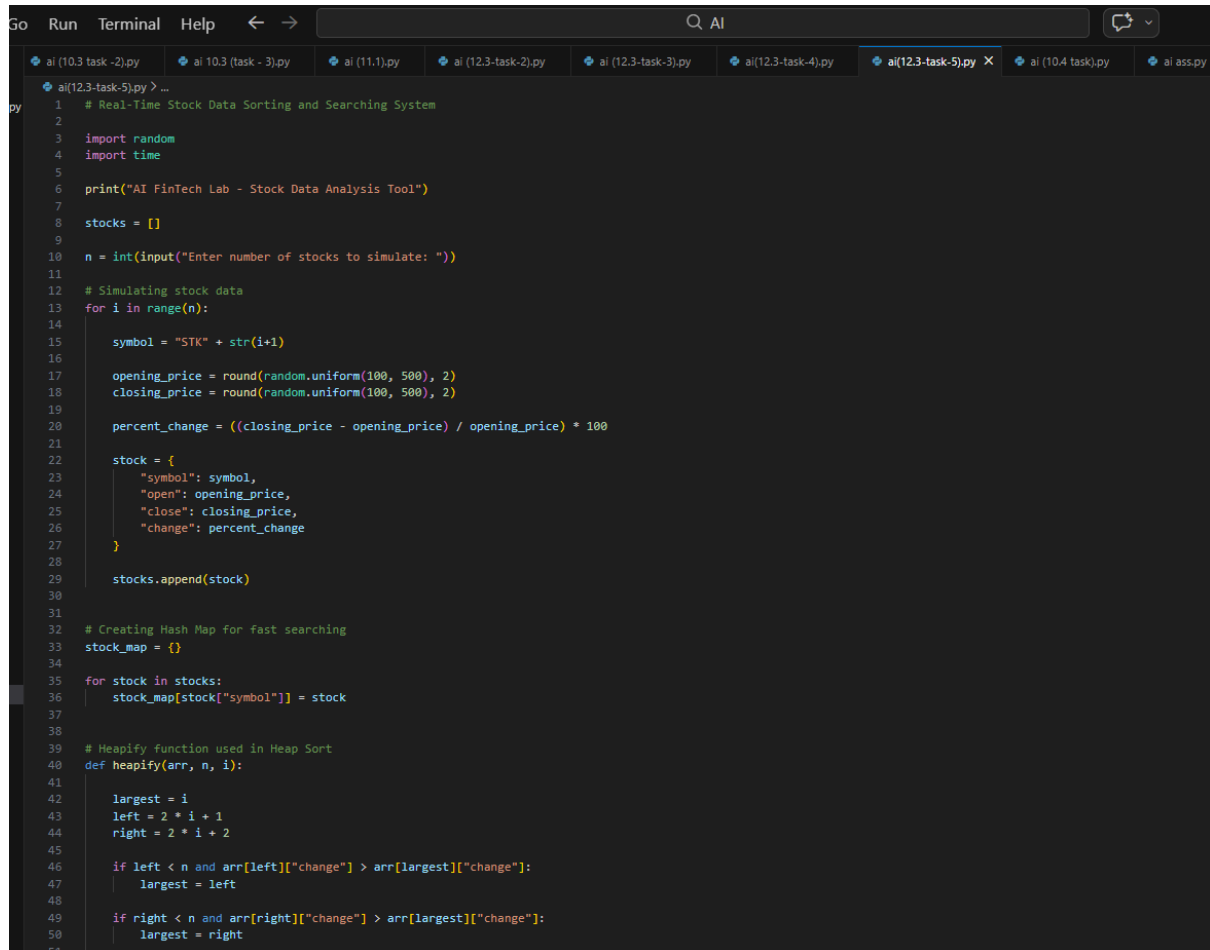
Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dict

lookups) and analyze trade-offs



```
Go Run Terminal Help ← → Q AI
ai(10.3 task-2).py ai(10.3 task-3).py ai(11.1).py ai(12.3-task-2).py ai(12.3-task-3).py ai(12.3-task-4).py ai(12.3-task-5).py X ai(10.4 task).py ai ass.py
ai(12.3-task-5).py > ...
1 # Real-Time Stock Data Sorting and Searching System
2
3 import random
4 import time
5
6 print("AI FinTech Lab - Stock Data Analysis Tool")
7
8 stocks = []
9
10 n = int(input("Enter number of stocks to simulate: "))
11
12 # Simulating stock data
13 for i in range(n):
14     symbol = "STK" + str(i+1)
15
16     opening_price = round(random.uniform(100, 500), 2)
17     closing_price = round(random.uniform(100, 500), 2)
18
19     percent_change = ((closing_price - opening_price) / opening_price) * 100
20
21     stock = {
22         "symbol": symbol,
23         "open": opening_price,
24         "close": closing_price,
25         "change": percent_change
26     }
27
28     stocks.append(stock)
29
30
31 # Creating Hash Map for fast searching
32 stock_map = {}
33
34 for stock in stocks:
35     stock_map[stock["symbol"]] = stock
36
37
38 # Heapify function used in Heap Sort
39 def heapify(arr, n, i):
40
41     largest = i
42     left = 2 * i + 1
43     right = 2 * i + 2
44
45     if left < n and arr[left]["change"] > arr[largest]["change"]:
46         largest = left
47
48     if right < n and arr[right]["change"] > arr[largest]["change"]:
49         largest = right
50
51
```



```
ai (10.3 task -2).py  ai 10.3 (task - 3).py  ai (11.1).py  ai (12.3-task-2).py  ai (12.3-task-3).py  ai(12.3-task-4).py  ai(12.3-task-5).py X
ai(12.3-task-5).py > ...
40 def heapify(arr, n, i):
52     if largest != i:
53         arr[i], arr[largest] = arr[largest], arr[i]
54         heapify(arr, n, largest)
55
56
57 # Heap Sort for ranking stocks by percentage change
58 def heap_sort(arr):
59
60     n = len(arr)
61
62     for i in range(n//2 - 1, -1, -1):
63         heapify(arr, n, i)
64
65     for i in range(n-1, 0, -1):
66         arr[i], arr[0] = arr[0], arr[i]
67         heapify(arr, i, 0)
68
69     return arr
70
71
72 # Searching stock using Hash Map
73 def search_stock(symbol):
74     """
75     Dictionary lookup allows very fast search.
76     Average time complexity is O(1).
77     """
78
79     return stock_map.get(symbol, None)
80
81
82 # Measuring Heap Sort performance
83 start = time.time()
84 sorted_stocks = heap_sort(stocks.copy())
85 heap_time = time.time() - start
86
87 print("\nStocks sorted by percentage change using Heap Sort")
88
89 for s in sorted_stocks:
90     print(s)
91
92
93 # Searching for a stock symbol
94 symbol = input("\nEnter stock symbol to search: ")
95
96 result = search_stock(symbol)
97
98 if result:
99     print("Stock Found:", result)
100 else:
101     print("Stock not found")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
```

```
ai(12.3-task-5).py > ...
101 | print("Stock not found")
102 |
103 |
104 | # Comparing with Python built-in sorting
105 | start = time.time()
106 | builtin_sorted = sorted(stocks, key=lambda x: x["change"])
107 | builtin_time = time.time() - start
108 |
109 | print("\nPerformance Comparison")
110 | print("Heap Sort Time:", round(heap_time, 6), "seconds")
111 | print("Python sorted() Time:", round(builtin_time, 6), "seconds")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Dell\Desktop\AI> cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
'--' 'c:\Users\Dell\Desktop\AI\ai(12.3-task-5).py'

AI FinTech Lab - Stock Data Analysis Tool
Enter number of stocks to simulate: 3

Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
AI FinTech Lab - Stock Data Analysis Tool
Enter number of stocks to simulate: 3

Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
```

Navigate to Command
Scroll to Previous Command (Ctrl+)
Scroll to Next Command (Ctrl+Dov

```
Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Heap Sort Time: 4.6e-05 seconds

Performance Comparison
Heap Sort Time: 4.6e-05 seconds
Heap Sort Time: 4.6e-05 seconds
Python sorted() Time: 3.7e-05 seconds
PS C:\Users\Dell\Desktop\AI> 
```

